

Lecture 03: The Linear Model I

Caltech CS156 Technical Study Guide

Gemini Notebook

September 18, 2026

Executive Summary

Lecture 03 transitions from general learning theory to practical algorithm design. It introduces high-level feature extraction using the ZIP code digit recognition problem, presents the **Pocket Algorithm** for non-separable data, derives the analytical **Pseudo-Inverse solution** for Linear Regression, demonstrates how Linear Regression can initialize classification algorithms, and introduces **Nonlinear Transformations** ($\mathbf{x} \rightarrow \mathbf{z}$) that extend linear machinery to complex decision boundaries while preserving linearity in weights $\mathbf{w}$.

1 Recap & Model Simplification

In previous lectures, the Generalization Bound was established via Hoeffding's Inequality and the Union Bound:

$$P(|E_{\text{in}}(g) - E_{\text{out}}(g)| > \epsilon) \leq 2Me^{-2\epsilon^2 N} \quad (1)$$

When the hypothesis set $\mathcal{H}$ contains M hypotheses, a large M loosens the bound. For infinite hypothesis spaces ($M = \infty$), this simple bound becomes non-informative. However, before developing full VC theory for $M = \infty$, we explore practical linear models.

2 ZIP Code Digit Recognition & Feature Extraction

Consider classifying handwritten digits from USPS postal mail (16×16 grayscale pixel images).

2.1 Raw Input vs. Feature Representation

- **Raw Input Representation:** An image contains $16 \times 16 = 256$ pixels. Including the bias term $x_0 = 1$, the input vector $\mathbf{x} \in \mathbb{R}^{257}$ requires 257 parameters ($w_0, w_1, \dots, w_{256}$). High-dimensional spaces lead to overfitting and severe generalization risks for small sample sizes N .
- **Feature Extraction:** Instead of raw pixels, domain knowledge is used to extract $d = 2$ interpretable high-level features:
 1. **Intensity** (x_1): Average pixel brightness across the image (e.g., digit '8' has higher intensity than digit '1').
 2. **Symmetry** (x_2): Measures horizontal and vertical reflectional symmetry (e.g., digit '1' is highly symmetric, while '5' is asymmetric).

By reducing dimension from 257 to 3 (x_0, x_1, x_2), the model drastically improves generalization while capturing essential class boundaries.

3 Handling Non-Separable Data: The Pocket Algorithm

Real-world data is rarely linearly separable due to mislabeling or feature overlap. The basic Perceptron Learning Algorithm (PLA) fails to terminate on non-separable data.

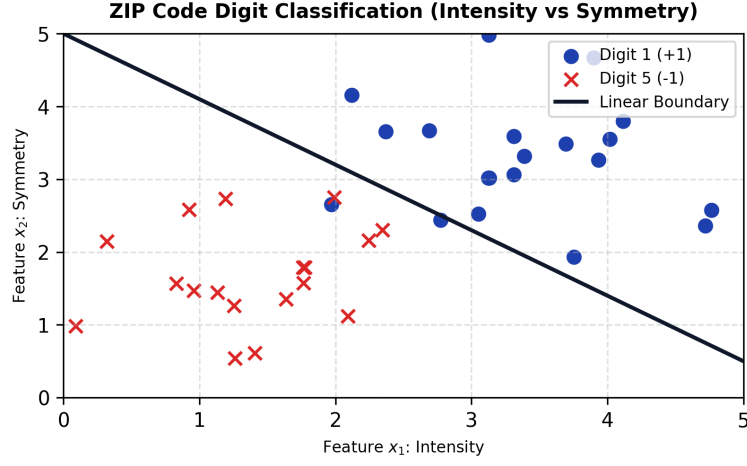


Figure 1: Classification of Digits 1 vs 5 in 2D Feature Space (Intensity vs Symmetry).

3.1 The Pocket Mechanism

To handle non-separable data, the **Pocket Algorithm** keeps track of the best weight vector encountered so far:

1. Initialize weight vector $\mathbf{w}(0)$ and set best weight $\mathbf{w}^* = \mathbf{w}(0)$ with in-sample error $E_{\text{in}}(\mathbf{w}^*)$.
2. At iteration t , run a standard PLA update on a misclassified point $(\mathbf{x}_n, y_n)$:

$$\mathbf{w}(t+1) = \mathbf{w}(t) + y_n \mathbf{x}_n \quad (2)$$

3. Compute $E_{\text{in}}(\mathbf{w}(t+1))$. If $E_{\text{in}}(\mathbf{w}(t+1)) < E_{\text{in}}(\mathbf{w}^*)$, update the "pocket": $\mathbf{w}^* \leftarrow \mathbf{w}(t+1)$.
4. Force termination at iteration T (e.g., $T = 1000$) and return $\mathbf{w}^*$.

4 Linear Regression & Analytical Optimization

Linear Regression predicts a continuous real-valued target $y \in \mathbb{R}$ (e.g., predicting credit limits or course grades).

4.1 Model Formulation & Squared Error

The linear hypothesis is defined as:

$$h(\mathbf{x}) = \sum_{i=0}^d w_i x_i = \mathbf{w}^T \mathbf{x} \quad (3)$$

The standard loss function is the **Mean Squared Error (MSE)**:

$$E_{\text{in}}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N (\mathbf{w}^T \mathbf{x}_n - y_n)^2 \quad (4)$$

4.2 Matrix Representation & Derivation

Expressing the dataset in matrix form where $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$ is the data matrix (with $x_0 = 1$ in the first column) and $\mathbf{y} \in \mathbb{R}^N$ is the target vector:

$$E_{\text{in}}(\mathbf{w}) = \frac{1}{N} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \frac{1}{N} (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) \quad (5)$$

To minimize $E_{\text{in}}(\mathbf{w})$, take the gradient with respect to $\mathbf{w}$ and set it to zero:

$$\nabla E_{\text{in}}(\mathbf{w}) = \frac{2}{N} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y}) = \mathbf{0} \implies \mathbf{X}^T \mathbf{X}\mathbf{w} = \mathbf{X}^T \mathbf{y} \quad (6)$$

4.3 The Pseudo-Inverse Solution

Assuming $\mathbf{X}^T \mathbf{X}$ is invertible (which holds when $N \gg d + 1$ and features are linearly independent), the analytic closed-form solution is:

$$\mathbf{w} = \mathbf{X}^\dagger \mathbf{y} \quad \text{where} \quad \mathbf{X}^\dagger = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \quad (7)$$

Here $\mathbf{X}^\dagger \in \mathbb{R}^{(d+1) \times N}$ is the **Moore-Penrose Pseudo-Inverse** of $\mathbf{X}$.

5 Linear Regression for Classification

Since binary targets $y \in \{-1, +1\}$ are real numbers, Linear Regression can be applied directly to classification problems:

1. Fit $\mathbf{w} = \mathbf{X}^\dagger \mathbf{y}$ using squared error on $y_n \in \{-1, +1\}$.
2. Predict class using $g(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x})$.

Key Insight: Linear Regression provides a fast, one-step analytical solution. While squared error penalizes points that are "too correct" (e.g., $y_n = +1$ but $\mathbf{w}^T \mathbf{x}_n = +5$), it serves as an excellent **initialization** for PLA/Pocket, drastically speeding up convergence.

6 Nonlinear Transformations ($\mathbf{x} \rightarrow \mathbf{z}$)

Linear models can learn complex non-linear decision boundaries via feature transformation.

6.1 The Mapping Concept

Define a non-linear mapping $\Phi: \mathcal{X} \rightarrow \mathcal{Z}$:

$$\mathbf{z} = \Phi(\mathbf{x}) = (1, \phi_1(\mathbf{x}), \phi_2(\mathbf{x}), \dots, \phi_d(\mathbf{x}))^T \quad (8)$$

For instance, a 2D quadratic transformation maps $\mathbf{x} = (x_1, x_2)$ to:

$$\mathbf{z} = (1, x_1, x_2, x_1^2, x_1 x_2, x_2^2)^T \in \mathbb{R}^6 \quad (9)$$

6.2 Preservation of Linearity

Crucially, the hypothesis remains **linear in the weight vector $\tilde{\mathbf{w}}$** :

$$h(\mathbf{x}) = \tilde{\mathbf{w}}^T \mathbf{z} = \tilde{\mathbf{w}}^T \Phi(\mathbf{x}) \quad (10)$$

All linear algorithms (OLS, PLA, Ridge) apply directly in $\mathcal{Z}$ -space without modification. The decision boundary is linear in $\mathcal{Z}$, but non-linear when mapped back to $\mathcal{X}$.